Photobooth Theme Asset Generation Guide

This guide outlines the process for creating new clothing styles and assets for the photobooth themes. Follow these three steps to ensure assets are correctly integrated into both the kiosk frontend and the receiver backend.

> [!NOTE] Detailed sample files for `data.js` and `clothes_mapping.json` from previous themes will be provided to you as a reference alongside this guide.

---

1. Prompt Design for AI Generation

The first step is designing high-quality prompts to generate the virtual try-on assets. (These can be generated using AI, such as Gemini, but don't make the prompt too long)

Framing Strategy:

- **Focus on Top Half:** Design prompts that focus on the upper body/torso. This ensures the generation is a **close-up** or **mid-shot**.
- **Avoid Pants:** If your prompt mentions "pants" or "shoes," the AI will likely trigger a **full-body shot**, which may not align with the close-up nature of the booth photos. (unless we want that)

Example Generation Prompt (Standalone Asset):

> "A professional studio product shot of a standalone female outfit, featuring a hybrid short-sleeved tunic in Emerald-Green silk with a vibrant Vermilion sash, topped with Polished Gold Shanwenjia (山纹甲) pauldrons. The shoulder guards are etched with Phoenix motifs that bleed into the fabric via gold-leaf embroidery, mixing leather textures with high-shine silk. The outfit is displayed to maintain a 3D shape, centered composition, high-quality fabric textures, cinematic lighting, isolated on a solid black background for easy transparency masking."

---

2. Testing for Consistency

Once you are happy with a specific clothes style output, **test it for consistency** by running it through the "Main Generation" prompt a few times.

Since the booth combines the user's face, a pose, and a background with your clothes style, you should verify the style holds up in the final composition.

Main Generation Sample Prompt (For Testing):

Key in the gender that the clothes was designed for, and a random pose and background for testing purposes, but fill in the clothes style with the standalone asset prompt you designed in step 1.

```
"Extreme Close up shot of a \"gender\" charismatic K-pop idol with \"pose\" for a
concept photo, dressed in \"clothes_style\".
The setting is \"background\". The image features a modern graphic design overlay:
```

```
a translucent frosted-glass border
at the top and bottom of the frame. Integrated into these borders is stylized
typography of the English text
\"Happy 50th Anniversary\". High-key cinematic lighting, idol posing, vibrant posh
black and gold color palette,
words are vibrant bright colors encased in a border contrasting the background, 8k
resolution."
```

---

3. Create Frontend Configuration (`data.js`)

For each new theme, you must **create a new `data.js` file**. This file registers your designs so they appear as selectable options in the kiosk UI.

File Location: `kiosk_frontend/static/themes/union50/js/data.js`

Example JS Format:

```
export const clothes = [
    {
        category: "unisex",
        code: "C11",
        title: "The Phoenix Guardian",
        description: "Emerald silk meets gold-plated phoenix armor and fiery
vermilion wraps",
        prompt: "A hybrid short-sleeved tunic in Emerald-Green silk with a vibrant
Vermilion sash, topped with Polished Gold Shanwenjia (山纹甲) pauldrons. The
shoulder guards are etched with Phoenix motifs that bleed into the fabric via
gold-leaf embroidery, mixing leather textures with high-shine silk.",
        image: "/static/themes/[theme_name]/images/clothes/the-phoenix-
guardian.png"
    },
    // Add more clothes here...
];
```

---

4. Create Receiver Mapping (`clothes_mapping.json`)

You must also **create a new `clothes_mapping.json` file**. This ensures the receiver backend (Linux DGX) knows how to light and describe the chosen outfit during the final high-quality render.

File Location: `dgx_photobooth/templates/themes/union50/clothes_mapping.json`

Example Mapping Format:

```
{
    "C11": {
        "desc": "short-sleeved tunic in Emerald-Green silk with a vibrant
```

```
  Vermilion sash",
          "lighting": "Emerald Green and Gold"
      }
  }
```

(Ensure this is a valid JSON object containing all your clothes codes).

Pro-Tip: Using AI for Metadata

You can use **Gemini** to automate the creation of this data. Simply provide the main clothing prompt and ask it to:

1. Generate a **High-End Title** (e.g., "The Phoenix Guardian").
2. Write a **Punchy Description** for the UI.
3. Extract a **Simplified Description** for the mapping `desc`.
4. Suggest **Complementary Lighting** (e.g., "Emerald Green and Gold").

Image Asset Management

1. **Rendering:** Generate a preview image using your clothing prompt.
2. **Format:** Save the photo asset as a **.png** file.
3. **Naming:** Use kebab-case (e.g., `the-phoenix-guardian.png`). **Ensure the filename matches the `image` path in `data.js` exactly.**